

学号：202300130183	姓名：宋浩宇
实验名称：实验二 中文分词基础实验	
1. 实验目的	
1. 理解中文分词的基本概念及其在自然语言处理中的作用。 2. 掌握中文分词与英文分词的区别。 3. 学习使用现有中文分词工具（如 jieba）进行文本分词。 4. 理解中文分词中的歧义问题与未登录词问题。 5. 尝试实现一个基于词典的最大正向匹配（FMM）分词算法。 6. 对比不同分词方法的结果并进行分析。	
2. 实验原理	
中文文本通常没有明显的词边界，因此需要通过分词算法将连续的汉字序列切分为词语序列。 例如： 我爱自然语言处理 → 我 / 爱 / 自然语言处理 常见的中文分词方法包括： 1. 基于词典的分词方法（如最大匹配算法） 2. 基于统计模型的方法（如 HMM、CRF） 3. 基于深度学习的方法（如 BiLSTM、BERT） 本实验主要介绍两种方法： （1）使用现有工具进行分词 （2）实现最大正向匹配（Forward Maximum Matching, FMM）算法 最大正向匹配（Forward Maximum Matching, FMM）是一种基于词典的中文分词方法，其基本思想是从句子的左侧开始扫描，每次尽可能匹配词典中最长的词语，并不断向右移动，直到完成整个句子的分词。该方法利用词典中已有的词语信息，通过贪心策略选择当前能够匹配到的最长词，从而将连续的汉字序列切分为若干词语。 在具体实现过程中，首先需要统计词典中所有词语的长度，并确定其中的最大词长。随后从句子的第一个字符开始处理，根据最大词长截取一个候选字符串，并判断该字符串是否存在于词典中。如果该字符串在词典中出现，则认为匹配成功，将该词语加入分词结果，并将扫描指针向右移动该词语的长度。如果该字符串不在词典中，则将匹配长度减少一个字符，再次尝试匹配。通过不断缩短匹配长度，算法会逐步寻找较短的词语。如果匹配长度逐渐缩短到仅剩一个字符时仍然无法在词典中找到对应词语，则将当前单个字符作为一个词输出，并将指针向右移动一个字符。随后继续对剩余的字符串执行相同的匹配过程，直到整个句子被处理完成。通过这种方式，算法始终优先选择当前位置能够匹配到的最长词语，因此称为最大正向匹配。例如，对于句子“南京市长江大桥”，若词典中包含“南京”“南京市”“长江”“长江大桥”“大桥”等词语，算法首先从句子开头开始匹配，并在词典中找到“南京市”，于是输出该词并向右移动三个字符。接着对剩余字符串“长江大桥”继续匹配，并在词典中找到“长江大桥”，最终得到分词结果“南京市 / 长江大桥”。这种方法实现简单、效率较高，因此常被用作中文分词的基础算法，但由于其仅依赖词典并采用贪心策略，在遇到歧义或未登录词时可能产生错误分词。	
3. 实验步骤与代码	
实验步骤：	

步骤 1: 安装并导入 jieba 库

```
import jieba
```

步骤 2: 使用 jieba 进行中文分词

```
import jieba
```

```
sentences = ["我爱自然语言处理", "研究生命起源", "南京市长江大桥", "乒乓球拍卖完了"]
```

```
for s in sentences:
```

```
    print("/".join(jieba.lcut(s)))
```

步骤 3: 实现最大正向匹配分词算法

步骤 4: 对比不同分词方法的结果。

代码:

```
import jieba
exp1_2_test_sentences = ["我爱自然语言处理",
                          "研究生命起源",
                          "南京市长江大桥",
                          "乒乓球拍卖完了",
                          "今天天气很好",
                          "小明硕士毕业于中国科学院",]

def load_exp1_2_data(path="./exp-1.2-data.txt"):
    jieba.load_userdict(path)
    dictionary = []
    with open(path, newline='') as file:
        for line in file:
            dictionary.append(line.strip())
    return dictionary

def get_exp1_2_model():
    dictionary = load_exp1_2_data()
    fmm = FMModel(dictionary)
    return fmm

def test_exp1_2():
    fmm = get_exp1_2_model()
    jieba_result = [jieba.lcut(i) for i in exp1_2_test_sentences]
    print(f"jieba 分词结果为:\n{jieba_result}")
    fmm_result = [fmm.lcut(i) for i in exp1_2_test_sentences]
    print(f"fmm 分词结果为\n{fmm_result}")
    three_bodies_fmm = FMModel(load_exp1_2_data("./《现代汉语词典》
(第 7 版).txt"))
    three_bodies_text = open("./[2006]《三
体》.txt").read().replace("\n", "")
    print(f"fmm 分词得到的三体 I 中词语数量为:
\n{len(three_bodies_fmm.lcut(three_bodies_text))}")

class FMModel:
    def __init__(self, dictionary):
```

```

self.dictionary = set(dictionary)
self.max_len = max([len(i) for i in dictionary])
def lcut(self, sentence):
    result = []
    i = 0
    while i < len(sentence):
        matched = False
        max_len = min(self.max_len, len(sentence) - i)
        for j in range(max_len, 0, -1):
            candidate = sentence[i:i + j]
            if candidate in self.dictionary:
                result.append(candidate)
                matched = True
                i += j
                break
        if not matched:
            result.append(sentence[i])
            i += 1
    return result
if __name__ == "__main__":
    test_exp1_2()

```

4. 实验结果（图表与输出）

输出结果为：

```

F:\Homework\自然语言处理作业\venv\Scripts\python.exe F:\Homework\自然语言处理作业
\第一周实验\实验二-中文分词基础实验.py
Building prefix dict from the default dictionary ...
Loading model from cache C:\Users\23676\AppData\Local\Temp\jieba.cache
jieba 分词结果为:
[['我', '爱', '自然语言处理'], ['研究', '生命', '起源'], ['南京市', '长江大桥'], ['乒乓球拍', '卖完
了'], ['今天天气', '很', '好'], ['小明', '硕士', '毕业', '于', '中国科学院']]
fmm 分词结果为
[['我', '爱', '自然语言处理'], ['研究', '生命', '起源'], ['南京市', '长江大桥'], ['乒乓球拍', '卖完
了'], ['今', '天', '天', '气', '很', '好'], ['小', '明', '硕', '士', '毕', '业', '于', '中国科学院']]
Loading model cost 0.312 seconds.
Prefix dict has been built successfully.
fmm 分词得到的三体 I 中词语数量为:
151548

```

图表结果为：

```

F:\Homework\自然语言处理作业\venv\Scripts\python.exe F:\Homework\自然语言处理作业\第一周实验\实验二-中文分词基础实验.py
Building prefix dict from the default dictionary ...
Loading model from cache C:\Users\23676\AppData\Local\Temp\jieba.cache
jieba分词结果为:
[['我', '爱', '自然语言处理'], ['研究', '生命', '起源'], ['南京市', '长江大桥'], ['乒乓球拍', '卖完了'], ['今天天气', '很', '好'], ['小明', '硕士', '毕业', '于', '中国科学院']]
fmm分词结果为
[['我', '爱', '自然语言处理'], ['研究', '生命', '起源'], ['南京市', '长江大桥'], ['乒乓球拍', '卖完了'], ['今', '天', '天', '气', '很', '好'], ['小', '明', '硕', '士', '毕', '业', '于', '中国科学院']]
Loading model cost 0.312 seconds.
Prefix dict has been built successfully.
fmm 分词得到的三体 I 中词语数量为:
151548

```

5. 结果分析

1. 中文分词和英文分词有什么区别？

英文分词相对简单，单词之间天然有空格作为分隔符，主要处理标点、大小写和特殊符号；而中文书写没有空格，分词需要识别字与字之间的隐含边界，涉及歧义消解和未登录词识别，难度更高。

2. 为什么句子“研究生命起源”会存在不同分词方式？

该句存在交集型歧义。“研究生”是一个常见词，但“研究/生命/起源”也是合理切分。两种切分在词典中都可能成立，导致歧义，需要依赖上下文或统计信息才能确定最优解。

3. 最大匹配算法的优点和缺点是什么？

优点：实现简单、速度快、对常见词汇覆盖较好。缺点：无法处理歧义（只选最长词，可能选错）、对未登录词（词典外词）无能为力、严重依赖词典质量和覆盖度。

4. 为什么词典方法难以处理未登录词？

词典方法完全依赖预定义的词表进行匹配，未登录词（新词、专有名词、缩写等）不在词典中，会被错误切分成单字或已知片段，无法识别完整语义单元，这是规则/词典方法的固有局限。

5. jieba 分词和自实现分词结果有什么差异？

以下内容为 LLM 生成：

自实现的是方法 FMM（正向最大匹配），属于纯词典规则方法。而 jieba 默认采用基于前缀词典和 HMM（隐马尔可夫模型）的混合策略，能更好地处理未登录词和歧义。差异主要体现在：jieba 对新词/人名识别更准，歧义消解更智能；FMM 实现简单但容易错切，尤其在未登录词和歧义句上表现较差。

6. 三体 I 中包含多少个词？请利用 FMM 实现分词方法，然后进行统计。

从算法运行结果上来看，是 151548 个词语

6. 实验总结

本实验我首先使用 jieba 对多个测试句子进行分词，然后自行实现了 FMM（正向最大匹配）算法，对比发现两者在测试语句上有些许出入，主要源自于 FMM 在未登录词处理中会出现严重错切，而 jieba 能正确识别。最后我用 FMM 基于《现代汉语词典》对《三体》全文进行分词，统计得到 151,548 个词语，验证了词典覆盖度直接影响分词效果。本次实验任务全部完成。